

Distributed SpMV on Multi-GPU Systems

Matteo Morellini – 268427

University of Trento

github.com/MatteoMorellini/GPU-Computing-Deliverable-2

matteo.morellini@studenti.unitn.it

Abstract—Sparse matrix–vector multiplication is memory-bound, and distributing it across GPUs adds a per-iteration exchange of remote vector entries whose cost is set by how the matrix is owned. This report builds one implementation on up to four NVIDIA A30 GPUs, fixing each design choice by measurement on nine matrices of diverse structure. CUDA-aware MPI improves host-staged transfers by 1.962×. Partition choice is matrix-dependent: 2D graph obtains the highest geometric-mean throughput, 113.5 GFLOP/s and 6.43× over cyclic, whereas 1D graph wins five of nine matrices and is the robust default at 97.3 GFLOP/s. Graph construction costs a 15.5 s geometric mean and breaks even after a median 4197 iterations. Communication reaches 41.9% of the iteration at four GPUs, and perfect overlap would still cap efficiency at 44.2%, making ownership rather than transport the dominant decision.

I. INTRODUCTION

Sparse matrix–vector multiplication (SpMV), $y = Ax$, performs 2 nnz floating-point operations while streaming sparse values, indices, and irregularly accessed vector entries, which makes it memory-bound on GPUs [1], [2]. Distributing the matrix adds two costs that a fast local kernel cannot remove: the exchange of remote vector entries at every iteration, and the choice of an ownership rule that keeps that exchange small without unbalancing the work.

This report builds one implementation on up to four GPUs as a sequence of controlled decisions, each measured against the configuration it replaces, with partitions compared through execution time, NNZ balance, and per-rank communication volume. The question is which combination of backend, input path, and partitioning strategy performs best at four GPUs, and which matrix properties determine that choice.

II. IMPLEMENTATION AND EXPERIMENTAL METHOD

A. Distributed SpMV and Data Movement

Each MPI rank drives one NVIDIA A30 GPU and converts its assigned COO entries to local CSR. Under 1D ownership a rank stores its output rows, its owned entries of x , and a compact ghost vector holding the distinct remote columns its matrix references; global column indices are remapped once to this local-plus-ghost indexing.

Setup builds a persistent exchange schedule, and each iteration packs the requested values on the GPU, exchanges them, and applies the adaptive CUDA CSR kernel. Host-staged MPI copies payloads through host memory, CUDA-aware MPI communicates device buffers directly, and an optional hybrid path keeps MPI for control and collectives while NCCL carries the repeated device payloads.

Algorithm 1 Adaptive CSR with long-row handling

```
1: CPU: split rows exceeding  $N$  into  $\lceil \text{nnz}_i / N \rceil$  chunks; group  
   the rest into blocks of  $\leq N$  nonzeros  $O(m)$   
2: for each block  $g$  in parallel do  
3:   if  $g$  is a long-row chunk then  
4:     CSR-Long: block-wide reduction  
5:   else if  $|\text{rows in } g| \geq \text{SMALLVALUE}$  then  
6:     CSR-Stream: stream into shared mem  
7:   else  
8:     CSR-Vector: block-wide reduction of the single row  
9:   end if  
10: end for
```

The 2D method follows Boman *et al.* [3], assigning nonzero (i, j) to a rank of a $P_r \times P_c$ grid from common row and column vertex parts; each iteration expands x , computes partial row products, and folds them to the output-row owners. The four-rank grid is 2×2 ; three-rank 2D cases are omitted because the grid collapses to the corresponding 1D layout.

B. Local Adaptive CSR Kernel

The local product uses CSR-Adaptive [2] (Algorithm 1), which dispatches each block to CSR-Stream or CSR-Vector at launch so that short rows do not idle a warp. The published version has no case for a row longer than N and falls back to one warp, which on FullChip’s 2.3M-nonzero row serializes the kernel; CSR-Long removes that dependence on maximum row length by chunking such rows across blocks and combining partial sums with `atomicAdd`. This makes the summation order nondeterministic, but it stays within the tolerance of Section II-D. The kernel is fixed across all experiments and is therefore a constant of the comparison.

C. Dataset and Inputs

Table I summarizes the nine square matrices after symmetry expansion. The portfolio is chosen for structural diversity: web graphs (`webbase-1M`, `webbase-2001`, `eu-2005`), synthetic scale-free graphs (`rmat_22`, `rmat_24`), circuit matrices (`FullChip`, `rajat31`), and a mesh-derived operator (`nlpkkt240`). Their per-row distributions span the regimes an ownership rule must handle, from near-regular rows (`nlpkkt240`) to a single row of 2.3M nonzeros (`FullChip`).

Three input paths are evaluated: rank 0 reads and distributes entries with `MPI_Scatterv`; distributed stdio independently reads line-aligned byte ranges; collective MPI-IO reads the same ranges with `MPI_File_read_at_all`.

TABLE I

SPARSE MATRIX STATISTICS ON PER-ROW NONZERO COUNT, AFTER SYMMETRY EXPANSION.

Matrix	Dim.	nnz	med	avg	std	max
webbase-1M	1.0M	3.1M	2	3.1	25.0	4.7K
eu-2005	863K	19.2M	16	22.3	29.0	7.0K
webbase-2001	118.1M	1.02B	3	8.6	26.0	3.8K
rmat_22	4.2M	37.7M	0	9.0	125.0	89.8K
rmat_24	16.8M	151.0M	0	9.0	159.0	207.7K
FullChip	3.0M	26.6M	6	8.9	1.8K	2.3M
rajat31	4.7M	20.3M	5	4.3	1.1	1.3K
Si41Ge41H72	186K	15.0M	37	80.9	127.0	662
nlpkt240	28.0M	774.5M	28	27.7	2.2	28

D. Platform, Validation, and Measurement

Experiments use one node with four NVIDIA A30 24-GB GPUs, 96 GB host memory, CUDA 12.3.2, and Open MPI 4.1.5; matrix and vector values use Float32. All runs use five warm-ups and 50 measured repetitions, whose total, communication, and computation times are separately reduced across ranks with `MP_I_MAX`; since their slowest ranks may differ, the two components are critical-path indicators rather than additive parts of the total. Reported times are arithmetic means with population standard deviations; that spread is a median 3.9% of the mean and reaches 33% on the smallest, most communication-bound cases, so per-matrix margins below roughly 10% are unresolved. The cyclic four-GPU reference, measured independently for the two sweeps, differs by 2.8% between them (5.58 and 5.738 ms below).

Throughput, speedup, and efficiency are

$$G_P = \frac{2 \text{ nnz}}{10^9 \bar{T}_P}, \quad S_P = \frac{G_P}{G_1}, \quad E_P = \frac{S_P}{P}, \quad (1)$$

where $\bar{T}_P$ is the mean total time after the rank-maximum reduction, and geometric means use the matrices valid for every method compared. The measured quantity is $\bar{T}_P$, the execution time of one SpMV, reported per matrix in milliseconds in Table III and as geometric means per backend and per partition in the text; every G_P , S_P , and E_P derives from it. Partition balance is $\text{nnz}_r / (\text{nnz} / P)$; communication volume counts Float32 values sent and received during expand and, for 2D methods, fold, excluding self-copies, setup, input, and final gathering. Two rank-level notions are kept distinct: *maximum-rank* is the largest per-rank value of a static quantity such as volume or NNZ, *critical rank* the rank that sets a measured `MP_I_MAX` time. Every volume in the V columns of Table II and in the text is maximum-rank; the few average-rank figures are labelled explicitly and never mixed into a comparison.

All gathered outputs pass against a double-precision CPU CSR reference with absolute and relative tolerances of 10^{-3} .

III. INCREMENTAL DESIGN SELECTION

A. Step 0: Baseline

The baseline combines rank-0 input, host-staged MPI, cyclic ownership, no NCCL, and the adaptive CUDA kernel; its one-GPU geometric-mean throughput is 79.2 GFLOP/s. Cyclic own-

TABLE II

FOUR-GPU PARTITION RESULTS PER MATRIX. FOR EACH PARTITION, V IS THE MAXIMUM-RANK COMMUNICATION VOLUME PER SpMV IN MiB AND G THE MEASURED THROUGHPUT IN GFLOP/s. BOLD MARKS THE BEST PARTITION OF EACH ROW IN EACH QUANTITY. TIME PER SpMV FOLLOWS AS $\bar{T}_4 = 2 \text{ nnz} / (10^9 G)$.

Matrix	Cyclic		1D blk		1D LRA		1D grph		2D blk		2D grph	
	V	G	V	G	V	G	V	G	V	G	V	G
FullChip	14.72	7.8	9.52	29.7	11.22	7.1	8.71	9.1	7.57	62.4	7.91	30.1
Si41Ge41H72	1.06	52.8	0.33	112.2	0.62	95.6	0.18	191.0	0.33	154.5	0.18	179.6
eu-2005	3.28	24.2	0.56	118.3	0.59	110.5	0.06	219.3	0.99	127.6	0.13	184.0
nlpkt240	132.7	20.0	54.95	79.9	49.87	85.4	1.56	387.4	54.94	84.3	2.08	401.1
rajat31	15.39	7.7	0.03	232.2	0.05	231.8	0.02	235.0	0.03	160.6	0.02	162.8
rmat_22	5.80	23.8	5.84	24.3	5.90	25.0	4.37	30.6	5.08	47.0	3.62	60.0
rmat_24	21.00	21.2	21.01	23.7	21.07	21.8	15.99	35.7	18.48	47.6	13.34	62.7
webbase-1M	0.39	23.7	0.13	52.1	0.22	43.5	0.01	65.9	0.30	40.0	0.01	47.2
webbase-2001	332.7	9.0	14.89	135.5	15.21	162.8	5.06	314.6	26.04	168.0	8.57	270.5
geo. mean	10.59	17.6	2.02	67.7	2.50	56.3	0.61	97.3	2.46	85.8	0.70	113.5

ership makes approximately $(P - 1)/P$ of irregular references remote: a deliberately communication-intensive reference.

B. Step 1: Host-Staged versus CUDA-Aware MPI

Replacing host-staged MPI with CUDA-aware MPI removes the explicit device-to-host and host-to-device staging of every payload. It improves all nine paired measurements at each process count: the geometric-mean time per SpMV falls from 8.63 to 2.73 ms at two GPUs, 11.11 to 5.15 ms at three, and 10.95 to 5.58 ms at four, that is by $3.1\times$, $2.1\times$, and $1.962\times$, and it is selected for all later experiments. The remaining limitation is that four-GPU efficiency stays at 5.7% under cyclic ownership: a faster backend moves the same volume faster but does not repair an ownership rule that makes most references remote.

C. Step 2: Input Method

The input path affects only setup. At two GPUs, distributed stdio gives the lowest geometric-mean setup time, 11.16 s against 14.61 s for rank-0 input and 22.75 s for collective MPI-IO; the gap is entirely read and parsing (rank maxima 10.41, 13.27, and 21.93 s), as COO-to-CSR conversion is identical across readers (6.74–6.78 s). Steady-state throughput spans only 37.3–38.1 GFLOP/s, approximately 2%, so no reader effect is evident and collective MPI-IO is retained as the memory-scalable alternative. The comparison is limited to two ranks on one filesystem.

IV. PARTITIONING STRATEGIES

With backend and reader fixed, ownership is the remaining design variable. Table II pairs, for every matrix and every timed partition at four GPUs, the maximum-rank communication volume V with the throughput G it produces, so that the per-matrix winners and the geometric means quoted below can be read directly. No partition is fastest everywhere: 1D graph wins five matrices, 2D graph three, and 2D block FullChip, so the scheme with the highest geometric mean is the best choice for only three of the nine.

A. One-Dimensional Ownership

Cyclic. Assigning rows round-robin balances work but ignores the sparsity pattern, so balance is near-perfect and locality is destroyed: the median most-loaded rank is 0.4% above ideal, while *webbase-2001* communicates 332.7 MiB on its heaviest rank. Cyclic is the weakest four-GPU partition, at a geometric-mean 5.738 ms per SpMV, and is the reference for Table II.

1D block. Contiguous row ranges keep neighbouring rows, and the vector entries they reference, on the same rank, at the risk of holding far more nonzeros than average. Both effects appear: the median maximum-rank volume falls from 14.72 to 5.84 MiB and the geometric-mean time per SpMV from 5.738 to 1.496 ms, a 3.84 $\times$ speedup over cyclic, while the busiest rank reaches 164.4% of average work on *FullChip*.

1D LRA. The long-row-aware variant targets matrices whose work is concentrated in a small region. From the longest row it identifies a prefix or suffix of $\lfloor 0.35n \rfloor$ rows, independently NNZ-balances that region and its complement into contiguous sub-blocks, and assigns one sub-block from each to every rank. This yields the best median balance, with the most-loaded rank rounding to 100.0% of ideal and LRA best balanced on eight of nine matrices, but its discontinuous ownership retains substantial remote-vector traffic and limits it to 1.797 ms per SpMV, a 3.19 $\times$ speedup that falls below plain block.

1D graph. METIS partitions the row graph with row NNZ as vertex weights, minimizing remote dependencies under a work-imbalance constraint, so both objectives are controlled at once rather than either being made exact. It attains the lowest median maximum-rank volume, 1.56 MiB, holds the worst observed load to 108.8% of average, and completes an SpMV in a geometric-mean 1.040 ms, 5.52 $\times$ faster than cyclic, making it the most robust partition method.

B. Two-Dimensional Ownership

2D block. Mapping the contiguous block partition onto a 2×2 process grid gives each rank a submatrix and partial rows that must be folded to the output-row owners: a shorter expand phase at the price of that fold. The median maximum-rank volume falls from 5.84 to 5.08 MiB, the geometric-mean time per SpMV from 1.496 to 1.181 ms and throughput rises from 67.7 to 85.8 GFLOP/s, 1.27 $\times$ over 1D block, and it is the best measured partition for *FullChip* (7.97 $\times$).

2D graph. Combining graph vertex parts with the same Cartesian nonzero ownership trades the reduced expand volume of the graph partition against that fold: across the nine complete pairs 1D graph is faster on five matrices and 2D graph on four, though the *Si41Ge41H72* (6.0%) and *nlpkkt240* (3.4%) margins lie within the combined standard deviation, and 2D graph holds the higher geometric mean, 0.892 ms per SpMV against 1.040 ms (6.43 $\times$ against 5.52 $\times$ over cyclic). Both directions occur: for *rmat_24*, 2D graph cuts maximum-rank communication from 15.99 to 13.34 MiB per rank and raises throughput from 35.7 to 62.7 GFLOP/s, whereas for *webbase-2001* maximum-rank communication rises from 5.06 to 8.57 MiB per rank and throughput falls from 314.6 to

TABLE III
STRONG SCALING FROM ONE TO FOUR GPUS WITH CUDA-AWARE MPI AND 1D GRAPH PARTITIONING. T_P IS THE MEAN EXECUTION TIME OF ONE SpMV ON P GPUS, $S_4 = T_1/T_4$ THE FOUR-GPU SPEEDUP, E_4 ITS PERCENT EFFICIENCY, AND GF/s THE FOUR-GPU THROUGHPUT. THE CYCLIC BASELINE REACHES $S_4 = 0.23$, $E_4 = 5.7\%$, AND 18.1 GFLOP/s.

Matrix	Time per SpMV (ms)				S_4	E_4	GF/s
	T_1	T_2	T_3	T_4			
FullChip	0.531	1.539	5.204	5.849	0.09	2.3	9.1
Si41Ge41H72	0.259	0.177	0.196	0.157	1.65	41.2	191.0
eu-2005	0.375	0.235	0.189	0.175	2.14	53.4	219.3
nlpkkt240	11.327	6.263	5.011	3.998	2.83	70.8	387.4
rajat31	0.435	0.266	0.203	0.173	2.51	62.9	235.0
rmat_22	1.797	1.238	2.020	2.470	0.73	18.2	30.6
rmat_24	11.585	6.979	8.302	8.458	1.37	34.2	35.7
webbase-1M	0.105	0.095	0.090	0.094	1.11	27.9	65.9
webbase-2001	16.312	9.597	7.217	6.484	2.52	62.9	314.6
geo. mean	1.278	0.981	1.088	1.040	1.23	30.7	97.3

270.5 GFLOP/s. Two-dimensional ownership pays only when the shorter expand critical path repays the fold.

C. Partition Construction Cost

Graph partitioning adds a one-time construction cost, geometric mean 15.5 s and up to 338 s, whose median break-even point over cyclic ownership at four GPUs is 4197 SpMV iterations. It is therefore worthwhile only when it is the faster partition for that matrix *and* the partition is reused for at least that many products. Input setup is reported separately and excludes construction, so a graph-partitioned run pays both.

V. SCALABILITY RESULTS

A. Strong Scaling of the Selected Configuration

The selected configuration is CUDA-aware MPI with 1D graph partitioning, taken as the robust default rather than the per-matrix winner (Table III). At four GPUs it raises geometric-mean throughput 5.4 $\times$ over the cyclic baseline, but scaling still degrades with P : efficiency falls from 65.1% at two GPUs to 39.2% at three and 30.7% at four, and the geometric-mean time per SpMV stops improving after two GPUs, 0.981 ms at two against 1.040 ms at four. Three of the nine matrices are slower at four GPUs than at two, *FullChip* most severely (1.539 to 5.849 ms, $E_4 = 2.3\%$), followed by *rmat_22* (1.238 to 2.470 ms) and *rmat_24* (6.979 to 8.458 ms). Selecting the fastest measured partition for each matrix instead of one default would give 138.9 GFLOP/s and $E_4 = 43.8\%$, an oracle bound on what partition choice alone can recover.

B. Communication Share and Overlap Bound

Figure 1 decomposes the efficiency loss in Table III. Communication's geometric-mean share of the iteration rises from 14.9% at two GPUs to 41.9% at four, from 0.146 to 0.435 ms, and computation drops from 0.651 to 0.355 ms. So total time stalls and the implementation becomes interconnect-bound between two and three GPUs. Only the CUDA-aware path is plotted: its timer brackets just the MPI calls, whereas the staged path's

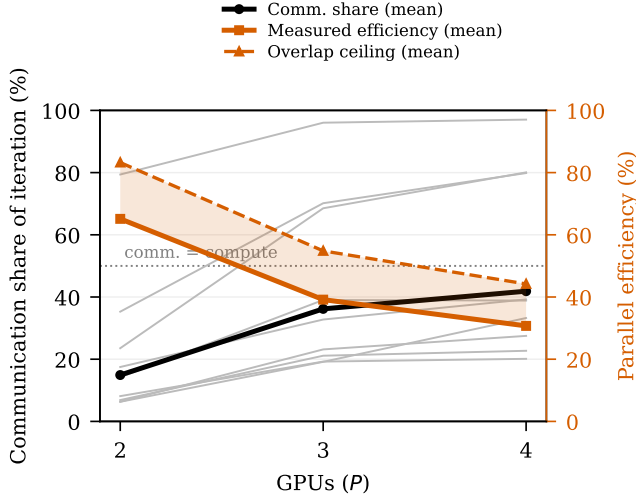


Fig. 1. **Communication share and overlap headroom of the selected configuration.** Left axis: communication as a share of the iteration, gray lines individual matrices. Right axis: geometric-mean measured efficiency and the perfect-overlap ceiling $E_P T_P / \max(T_{\text{comm}}, T_{\text{compute}})$.

uncounted gather and host-to-device copy are a median 71.4% of its four-GPU iteration, making the two incomparable.

Per-matrix shares explain the non-monotone rows of Table III: the four matrices with the lowest four-GPU share (20.1–33.2%) are exactly the four retaining $E_4 > 50\%$, whereas *FullChip* spends 97.0% of its iteration communicating and 2.6% on arithmetic; a communication problem that no kernel or NNZ-balance fix would address. Perfect overlap (Fig. 1) is worth at most 1.44 $\times$ at four GPUs, capping E_4 at 44.2%, not enough to close the gap to ideal scaling.

Hybrid MPI+NCCL, with NCCL carrying only ghost, fold, and final-gather payloads, was also run on the four-GPU 2D graph path: across the nine identical-partition pairs it is the slower backend, 105.7 against 113.5 GFLOP/s geometrically (0.932 $\times$), winning three of them.

C. Weak Scaling and Capacity

Weak scaling uses random synthetic CSR matrices with 262,144 rows and 8,388,608 nonzeros per GPU: every row holds exactly 32 nonzeros, whose column indices are drawn uniformly at random over the whole global column range, from a fixed seed (20260721). Computation is flat, 0.363 ms at one GPU against 0.380 ms at four, as intended by the fixed per-rank problem; maximum-rank communication instead grows from a negligible 0.07 μ s to 0.282, 1.567, and 3.605 ms at two, three, and four GPUs, an order of magnitude above the local work it accompanies. Total time therefore follows communication rather than computation, rising 10.7 $\times$ from 0.372 to 3.994 ms, so throughput peaks at 50.3 GFLOP/s at two GPUs and falls to 16.8 GFLOP/s at four, an efficiency of 9.3%. The most-loaded rank holds only 1.03 $\times$ the average NNZ count, confirming that communication growth, rather than work imbalance, is the limit.

The largest completed real input is *webbase-2001*, whose validated four-rank execution fits within the 96-GB host allocation: compact ghost vectors avoid replicating the full vector, rank-0 input remains the capacity bottleneck for larger matrices.

VI. CROSS-MATRIX DISCUSSION

No single static metric predicts throughput. Cyclic and LRA ownership show that near-perfect NNZ balance coexists with large remote traffic, and 1D block the converse; graph partitioning succeeds by constraining both adequately rather than making either exact, the decisive change usually being communication. Maximum-rank volume, not the average, is the statistic that tracks the measured time, and *FullChip* separates them: moving from 1D graph to 2D graph *raises* its average-rank volume from 6.64 to 7.35 MiB, yet *lowers* the maximum-rank volume from 8.71 to 7.91 MiB, and it is the latter that tracks the measured communication time and the throughput gain. *webbase-2001* moves both upward, average-rank 3.79 to 7.21 MiB, maximum-rank 5.06 to 8.57 MiB, and loses throughput. Message structure, synchronization, and the 2D fold likely explain the residual mismatch.

Mesh structure indicates locality and degree regularity, yet does not by itself determine the partitioning strategy. It is consistent with *nlpkt240*, where 1D graph gives essentially exact balance and a maximum-rank volume of 1.56 MiB. The non-mesh group is not uniform: the web graphs lose under an unnecessary fold, whereas the symmetric scale-free *rmat_24* gains from 2D graph, agreeing qualitatively with Boman *et al.* [3], and *FullChip*’s 2.3M-nonzero row leaves any 1D row assignment either imbalanced or communication-heavy, consistent with its gain from both two-dimensional modes.

VII. CONCLUSION

Partitioning, not the backend or the reader, is the dominant decision: CUDA-aware MPI improves every paired measurement yet leaves four-GPU efficiency at 5.7% under cyclic ownership, and the readers differ by approximately 2% in steady state. No partition is universally best: 1D graph wins the most individual matrices and is the robust default at 97.3 GFLOP/s and $E_4 = 30.7\%$, while 2D graph holds the higher geometric mean (6.43 $\times$ over cyclic) because its gains on the four matrices it wins outweigh its losses on the other five. Communication limits four-GPU scaling: it accounts for 41.9% of the iteration, and even perfect overlap would cap efficiency at 44.2%. In practice, choose 1D graph by default, check per-rank communication before adopting a 2D grid, and build a graph partition only when the same partition will be reused for several thousand SpMV products.

REFERENCES

- [1] N. Bell and M. Garland, “Implementing sparse matrix-vector multiplication on throughput-oriented processors,” in *Proc. SC*, 2009, pp. 1–11.
- [2] J. L. Greathouse and M. Daga, “Efficient sparse matrix-vector multiplication on GPUs using the CSR storage format,” in *Proc. SC*, 2014, pp. 769–780.
- [3] E. G. Boman, K. D. Devine, and S. Rajamanickam, “Scalable matrix computations on large scale-free graphs using 2D graph partitioning,” in *Proc. SC*, 2013, Art. no. 50, doi: 10.1145/2503210.2503293.